Project Report Format

1. PROJECT TITLE

Quotes Recommendation Chatbot using NLP

2. PROJECT DESCRIPTION

- System Overview
  A conversational chatbot that recommends motivational and inspirational quotes based on user intent using Natural Language Processing (Rasa). The UI is built with Streamlit.

- Problem Domain
  Users often waste time searching for relevant quotes online. Existing platforms are static and non-interactive.

- Core Technology Used
  1. Python
  2. Rasa (NLU + Core)
  3. Streamlit (Web UI)
  4. REST API
  5. SQLite (optional for favorites)

- Key Capabilities
  1. Intent detection from user text
  2. Category-based quotes (motivation, love, success, etc.)
  3. Emotional support responses
  4. Favorite quotes feature
  5. Interactive chat interface

- Target Users
  1. Students under academic stress
  2. Professionals seeking motivation
  3. Users looking for inspirational content

- Real-world Relevance
  Promotes mental well-being and demonstrates practical use of conversational AI.

3. APPLICATION SCENARIOS / USE CASES

- Enterprise usage
  Customer engagement bots for wellness platforms.

- End-user usage
  Individuals interacting with chatbot for motivation.

- Educational usage
  Students learning NLP and conversational AI.

- Industrial usage
  Integration into mental wellness or productivity apps.

4. TECHNICAL ARCHITECTURE OVERVIEW

- High-level Architecture Diagram
  User → Streamlit UI → REST API → Rasa NLP Engine → Response → UI

- Component Interaction Diagram
  1. Streamlit captures input
  2. REST API sends message to Rasa

3. Rasa processes intent
4. Response returned to UI

- Data / Request Flow
  1. User enters message
  2. Message sent to Rasa server
  3. Intent classified using DIET classifier
  4. Response generated
  5. Displayed in UI

Layers

- Frontend
  - Streamlit UI

- Backend
  - Python + Rasa

- Database
  - SQLite (favorites)

- External Services
  - REST API communication

- Deployment
  - Local server

5. PREREQUISITES

5.1 Software Requirements

- Python 3.9+
- VS Code / PyCharm
- Git
- Rasa framework

5.2 Libraries / Frameworks / Dependencies

- rasa
- streamlit
- requests
- scikit-learn
- tensorflow (Rasa dependency)

5.3 Hardware Requirements (If Applicable)

- Laptop/Mobile

6. PRIOR KNOWLEDGE REQUIRED

- Python programming
- Basic NLP concepts
- REST API basics
- YAML configuration
- Machine learning basics

7. PROJECT OBJECTIVES

- Technical objectives

  Build a chatbot capable of understanding user intent.

- Performance objectives
  Provide quick responses (<2 seconds).

- Deployment objectives
  Web-based chatbot accessible through browser.

- Learning outcomes
  Understand conversational AI and NLP frameworks.

8. SYSTEM WORKFLOW

1. User opens chatbot UI
2. User enters message
3. Streamlit sends message to Rasa API
4. Rasa processes intent
5. Response selected
6. Response displayed in UI

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition
Users need quick access to motivational quotes through interactive conversation.

9.2 Functional Requirements
- Chat interaction
- Intent detection
- Quote recommendation
- Favorite quotes feature

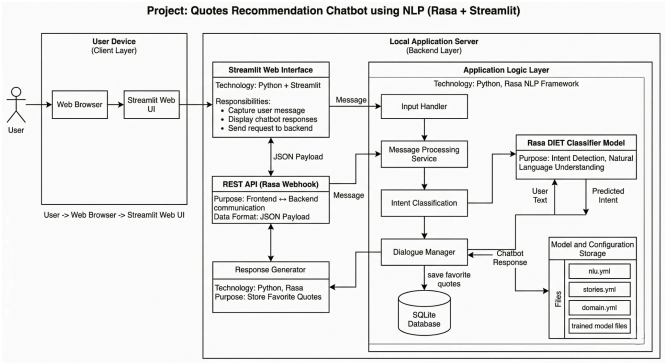
9.3 Non-Functional Requirements

- Usability
- Performance
- Reliability

9.4 System Design Decisions
Use Rasa NLP for conversation management.

9.5 Technology Stack Justification
Rasa provides robust NLP pipelines for chatbot systems.

Include:
- Architecture diagram



10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

Development Setup

- conda create -n rasa_env python=3.9
- conda activate rasa_env
- pip install rasa streamlit

Project Structure Creation
- rasa init

Configuration

- config.yml
- domain.yml
- nlu.yml
- Stories.yml

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

11.1 Feature 1 – Chat Interface
Implemented using Streamlit.

11.2 Feature 2 – Intent Classification
Rasa DIET classifier processes user text.

11.3 Feature 3 – Quote Recommendation
Quotes mapped to intents in domain.yml.

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

12.1 Component Integration
Streamlit UI connected with Rasa REST API.

12.2 Performance Optimization
Optimized NLU training data.

12.3 Security Enhancements
Input validation.

12.4 Error Handling & Validation
Fallback responses.

13. MILESTONE 5: TESTING & VALIDATION

13.1 Test Cases

Test Case ID	Input	Expected Output	Status
TC1	hi	Greeting message	Pass
TC2	Motivate me	Motivational quote	Pass
YC3	Love quote	Love quote	Pass
TC4	bye	Goodbye message	Pass

Testing
13.2 Unit testing
13.3 Integration testing
13.4 User acceptance testing

13.5 Performance Metrics
Response time < 2 seconds

If AI/ML:

- Accuracy = 100%
- Precision = 1.0
- Recall = 1.0
- F1 Score = 1.0

14. DEPLOYMENT

14.1 Deployment Architecture
Local server deployment.

14.2 Hosting Platform
Local system / Streamlit run.

14.3 Deployment Steps
rasa train
rasa run --enable-api
streamlit run app.py

14.4 Production Considerations
Cloud hosting possible using AWS or Heroku.

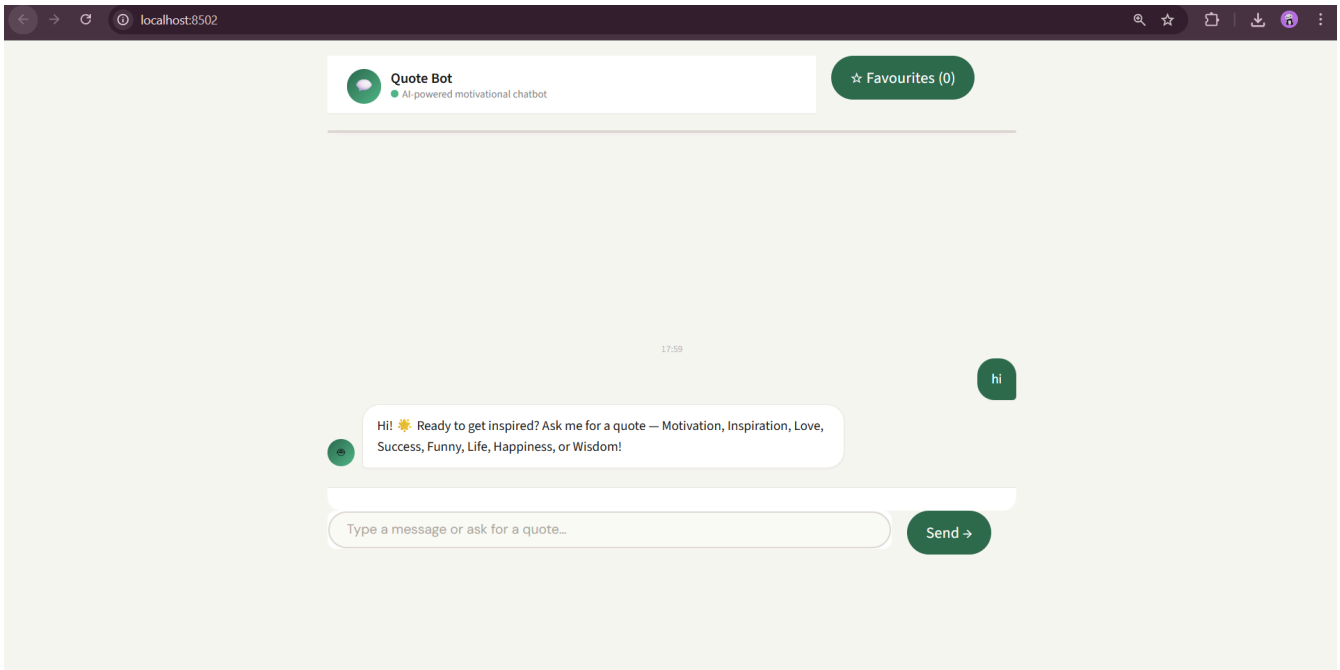
15. PROJECT STRUCTURE

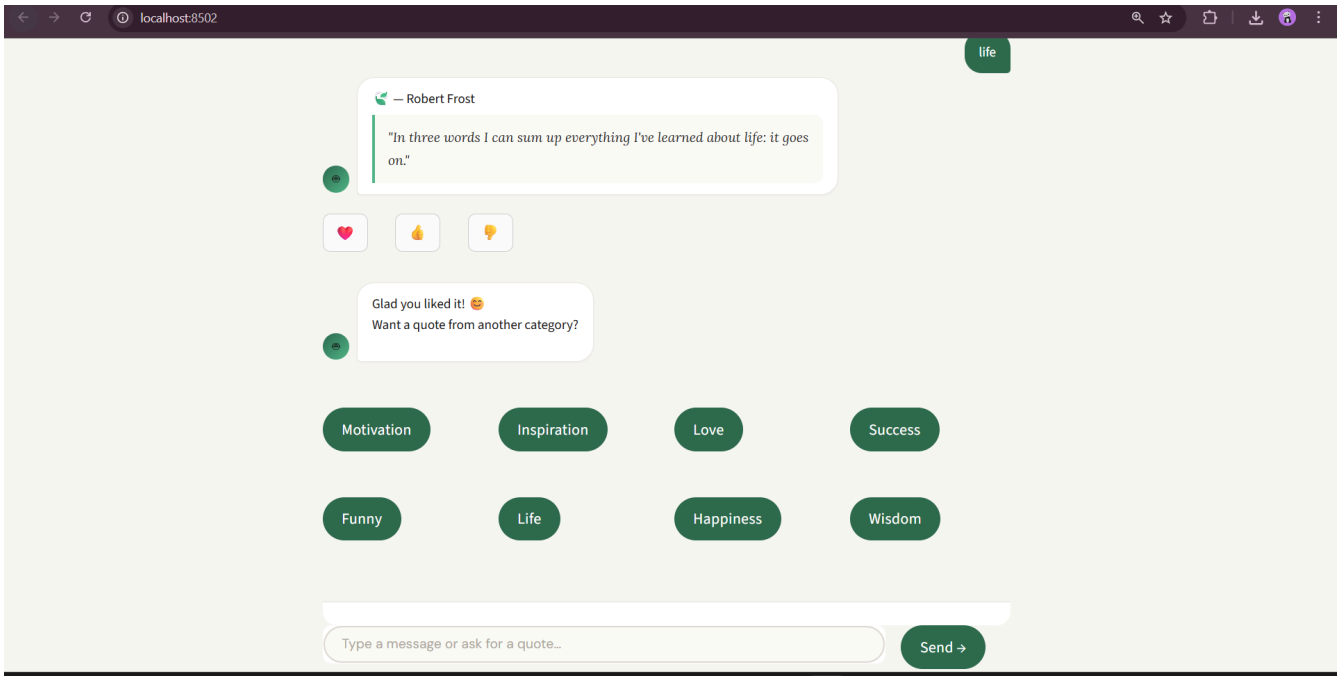
```
quotes-recommendation-chatbot-nlp/
│
├── actions/                      # Custom actions for Rasa
│   ├── __init__.py
│   └── actions.py
│
├── data/                         # Rasa training data
│   ├── nlu.yml                   # User intents and training examples
│   ├── rules.yml                 # Hardcoded conversation rules
│   └── stories.yml               # Conversation flows and logic
│
├── models/                       # Trained chatbot models
│   └── *.tar.gz                  # Auto-generated model files
│
├── tests/                        # Automated testing data
│   └── test_stories.yml          # End-to-end test scenarios
│
├── app.py                        # Streamlit web frontend (Main UI)
├── config.yml                    # Rasa NLU pipeline and Core policies
├── credentials.yml               # REST API and channel configuration
├── domain.yml                    # Chatbot universe (Intents, Responses, Entities)
├── endpoints.yml                 # Service endpoints (Action server, etc.)
│
├── requirements.txt              # Required Python dependencies
├── Prerequisites.txt             # Setup and installation instructions
└── README.md                     # Comprehensive project documentation
```

16. RESULTS

- Chatbot successfully responds to user queries
- Provides quotes based on categories

- Supports interactive conversation
- Screenshots





17. ADVANTAGES & LIMITATIONS

- **Advantages**
  - Interactive conversational interface
  - Personalized quote recommendation
  - Easy web access

- **Limitations**
  - Limited training data
  - No voice interface
  - Basic personalization

18. FUTURE ENHANCEMENTS

- Sentiment analysis
- Voice interaction
- Mobile application
- Multi-language support
- Cloud deployment

19. CONCLUSION

The project successfully demonstrates the implementation of a conversational AI chatbot using Rasa NLP and Streamlit. The chatbot effectively understands user intent and provides motivational quotes through an interactive web interface.

20. APPENDIX

- **Source Code**
- **Configuration Files**
- **Dataset Details (if applicable)**
- **API Documentation**
- **GitHub Repository Link** https://github.com/omreddy45/quotes-recommendation-chatbot-nlp
- **Live Demo Link** https://drive.google.com/drive/folders/1x6nzOf59Gj4IqplDae8Cy1cfBGFdnR1w?usp=sharing